

Databases for Big Data

Exercises: Parallel joins

I.Savnik, 12/12/2025.

Example database Sales

The following database contains two entity tables, Customer and Product, which store descriptive information about business entities. A Sales relation links these entities and records measurable events (e.g., quantities sold), forming a simple OLAP schema suitable for analytical queries.

```
CREATE TABLE Customer (
  cid    INT PRIMARY KEY,
  name   TEXT NOT NULL,
  region TEXT
);
```

```
CREATE TABLE Product (
  pid     INT PRIMARY KEY,
  name    TEXT NOT NULL,
  category TEXT
);
```

```
CREATE TABLE Sales (
  sale_id INT PRIMARY KEY,
  cid     INT NOT NULL,
  pid     INT NOT NULL,
  quantity INT NOT NULL,
  FOREIGN KEY (cid) REFERENCES Customer(cid),
  FOREIGN KEY (pid) REFERENCES Product(pid)
);
```

Here are the examples of the tables that are instances of the above relations.

Product

pid	name	category
1	Laptop	Electronics
2	Desk Chair	Furniture
3	Coffee Maker	Appliances
4	Notebook	Stationery
5	Headphones	Electronics
6	Desk Lamp	Furniture
7	Water Bottle	Accessories

Customer

cid	name	region
1	Alice Brown	West
2	Bob Smith	East
3	Carla Torres	North
4	Daniel Kim	South
5	Eva Johnson	Central

Sales

sale_id	cid	pid	quantity
1	1	1	2
2	1	4	5
3	2	3	1
4	2	7	3
5	3	2	2
6	3	5	1
7	4	1	1
8	4	6	2
9	5	3	4
10	5	7	2

Queries

Query 1.

```
select P.pid, P.category
from Product P, Sales S
where P.name = 'Notebook'
and    P.pid=S.pid
and    S.quantity>10;
```

Query 2.

```
select C.name, P.name, S.quantity
from Product P, Sales S, Customer C
where C.cid=S.cid
and    P.pid=S.pid
and    C.region='West';
```

Exercises

Exercise 1.

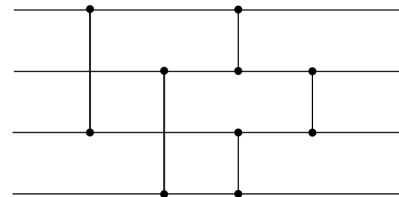
Demonstrate the execution of no partitioning hash join on Product $\bowtie$ Sales. Use the hash function that constructs the value from the least significant bits of the join key.

Exercise 2.

Sketch the execution of the Radix hash join on the example of join Customer $\bowtie$ Sales. Use the hash function that constructs the value from the least significant bits of the join key. Explain how the presented join can be parallelized.

Exercise 3.

Describe how the sorting network expressed in Knuth's notation works. Present the algorithmic version of sorting networks that does not include any conditional branches.



Exercise 4.

- Present the key ideas of the bitonic merge algorithm for merging sorted runs of values.
- Describe how the bitonic networks of size 8 can be combined to sort larger runs of values.

Exercise 5.

Demonstrate the execution of Multi-way sort-merge join on Product $\bowtie$ Sales.

Exercise 6.

Demonstrate the execution of Massively parallel sort-merge join on Customer $\bowtie$ Sales.

- Use the Basic MPSM join algorithm.
- Use the Range-partitioned MPSM join algorithm.